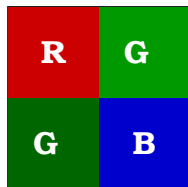


Interpolación Bicuada y Bicúbica para Patrón Bayer RGGB

Brandon Marquez Salazar

Procesamiento Digital de Señales en Tiempo Real
Universidad de Guanajuato



Patrón Bayer RGGB 2×2

24 de diciembre de 2025

Índice general

1	Introducción	2
1.1	Contexto del Problema	2
1.2	Problema de Interpolación	2
1.3	Objetivos del Estudio	2
1.4	Organización del Documento	2
2	Métodos	3
2.1	Patrón Bayer RGGB	3
2.2	Interpolación Lineal	3
2.3	Interpolación Bicuada	4
2.4	Interpolación Bicúbica	5
2.5	Proceso Bidimensional	6
3	Resultados	6
3.1	Implementación Práctica	6
3.2	Complejidad Computacional	6
3.3	Calidad de Interpolación	6
3.4	Problemas en Bordes	6
3.5	Vecindarios Utilizados	7
4	Discusión	7
4.1	Comparación de Métodos	7
4.1.1	Interpolación Lineal	7
4.1.2	Interpolación Bicuada	7
4.1.3	Interpolación Bicúbica	7
4.2	Consideraciones Prácticas	8
4.3	Limitaciones y Mejoras	8
4.3.1	Limitaciones Identificadas	8
4.3.2	Mejoras Posibles	8
5	Conclusión	8
5.1	Conclusiones Principales	8
5.2	Contribuciones del Trabajo	9
5.3	Recomendaciones de Uso	9
5.4	Trabajo Futuro	9
5.5	Consideraciones Finales	10
A	Códigos Fuente	10
A.1	Implementación en MATLAB	10

1 Introducción

1.1 Contexto del Problema

Los sensores de imagen en cámaras digitales utilizan filtros de color para capturar información cromática. El patrón Bayer, desarrollado por Bryce Bayer en 1976, es el arreglo más común, donde cada fotosito del sensor captura únicamente un canal de color (rojo, verde o azul).

1.2 Problema de Interpolación

Dada una matriz 2×2 del patrón Bayer:

$$\begin{bmatrix} R & G \\ G & B \end{bmatrix}$$

Es necesario estimar los valores faltantes en cada posición para reconstruir tres canales completos (RGB) con la misma resolución espacial. Este proceso se denomina **demosaicing** o interpolación de Bayer.

1.3 Objetivos del Estudio

1. Implementar algoritmos de interpolación bicuadrada para el patrón Bayer RGGB
2. Implementar algoritmos de interpolación bicúbica para el patrón Bayer RGGB
3. Comparar los resultados con métodos de interpolación lineal tradicional
4. Analizar la complejidad computacional de cada método
5. Visualizar los procesos mediante diagramas y gráficos explicativos
6. Proporcionar implementaciones prácticas en MATLAB para uso educativo

1.4 Organización del Documento

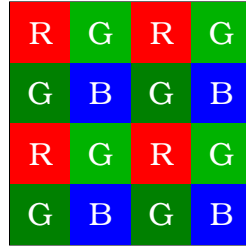
Este documento se estructura de la siguiente manera: la Sección 2 describe los métodos matemáticos utilizados; la Sección 3 presenta los resultados obtenidos; la Sección 4 analiza los hallazgos; y finalmente, la Sección 5 ofrece conclusiones y trabajo futuro. Los archivos de código fuente se encuentran en el Apéndice.

2 Métodos

2.1 Patrón Bayer RGGB

El patrón Bayer RGGB organiza los filtros de color en una matriz donde:

- Las filas pares comienzan con rojo (R) y verde (G) alternados
- Las filas impares comienzan con verde (G) y azul (B) alternados



Patrón Bayer RGGB 4×4

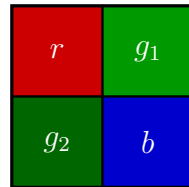
Figura 1: Distribución de canales en patrón Bayer RGGB

2.2 Interpolación Lineal

La interpolación lineal es el método más simple, usando el promedio de vecinos inmediatos:

$$P_{\text{interp}} = \frac{\sum_{k=1}^K P_k}{K}$$

Patrón Bayer 2×2



Píxel RGB

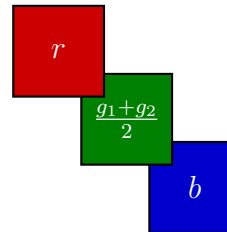


Figura 2: Proceso de demosaicado bilineal para patrón Bayer RGGB. Los canales de color se superponen para formar el píxel RGB final.

2.3 Interpolación Bicuadrada

Utiliza tres puntos para ajustar una parábola:

$$P(x) = ax^2 + bx + c$$

Para puntos equidistantes en $x \in \{-2, 0, 2\}$ (separados por 2 unidades, como en el patrón Bayer), la fórmula de interpolación para el punto intermedio en $x = 1$ es:

$$P(1) = \frac{3}{8}P(-2) + \frac{3}{4}P(0) - \frac{1}{8}P(2)$$

En coordenadas de imagen, para el canal rojo en posición $(2k, j)$:

$$R_{2k,j} = \frac{3}{8}R_{2k-2,j} + \frac{3}{4}R_{2k,j} - \frac{1}{8}R_{2k+2,j}$$

Patrón Bayer Original

R	G	R
G	B	G
R	G	R

$\Rightarrow$

Canal R

R	0	R
0	0	0
R	0	R

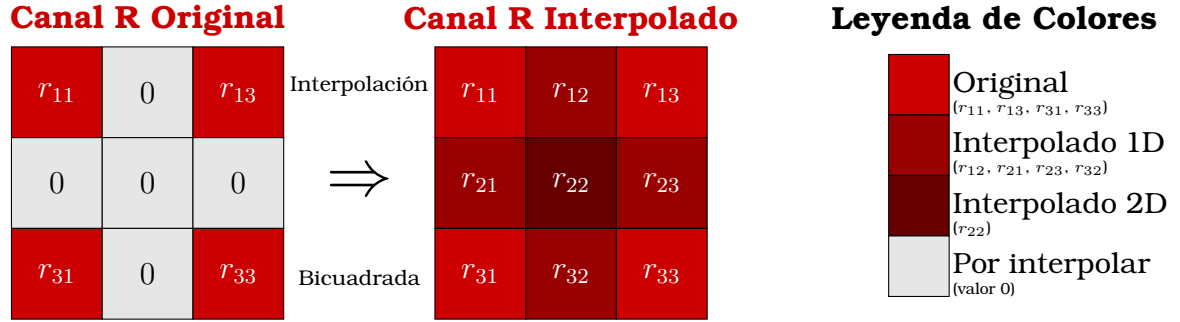
Canal G

0	G	0
G	0	G
0	G	0

Canal B

0	0	0
0	B	0
0	0	0

Figura 3: Separación de canales en patrón Bayer 3×3. El patrón original se descompone en tres matrices que muestran la información de cada canal por separado. Los ceros (gris) indican ausencia de información en esa posición para el canal correspondiente.



Fórmulas de Interpolación Bicuadrada

$$\begin{aligned}
 r_{12} &= \frac{3}{8}r_{11} + \frac{3}{4}r_{13} - \frac{1}{8}r_{15} & r_{23} &= \frac{3}{8}r_{13} + \frac{3}{4}r_{33} - \frac{1}{8}r_{53} \\
 r_{21} &= \frac{3}{8}r_{11} + \frac{3}{4}r_{31} - \frac{1}{8}r_{51} & r_{32} &= \frac{3}{8}r_{31} + \frac{3}{4}r_{33} - \frac{1}{8}r_{35} \\
 r_{22} &= \frac{9}{64}r_{11} + \frac{9}{32}r_{13} + \frac{9}{32}r_{31} + \frac{9}{16}r_{33} - \frac{3}{64}r_{15} - \frac{3}{64}r_{51}
 \end{aligned}$$

Figura 4: Interpolación bicuadrada para el canal rojo (R) en patrón Bayer. Arriba se muestra la transformación de la matriz original (con valores conocidos en rojo oscuro y ceros en gris) a la matriz interpolada (todos los valores completos, diferenciando entre originales, interpolados 1D e interpolados 2D). La leyenda, alineada verticalmente con las matrices, explica los colores utilizados. Debajo se presentan las fórmulas de interpolación bicuadrada con espaciado uniforme, distribuidas en dos columnas para una presentación clara y ordenada.

2.4 Interpolación Bicúbica

Utiliza cuatro puntos para ajustar un polinomio cúbico:

$$P(x) = ax^3 + bx^2 + cx + d$$

Con puntos en $x \in \{-2, 0, 2, 4\}$, usando el kernel cúbico de Catmull-Rom:

$$P(1) = -\frac{1}{16}P(-2) + \frac{9}{16}P(0) + \frac{9}{16}P(2) - \frac{1}{16}P(4)$$

Para el canal verde en posición $(2k+1, 2\ell)$:

$$G_{2k+1,2\ell} = -\frac{1}{16}G_{2k-3,2\ell} + \frac{9}{16}G_{2k-1,2\ell} + \frac{9}{16}G_{2k+3,2\ell} - \frac{1}{16}G_{2k+5,2\ell}$$

2.5 Proceso Bidimensional

La interpolación 2D se realiza en dos pasos independientes: primero se interpola verticalmente y luego horizontalmente (o viceversa). Este enfoque separable reduce la complejidad computacional respecto a una interpolación 2D directa.

3 Resultados

3.1 Implementación Práctica

Los algoritmos fueron implementados en MATLAB y probados con imágenes sintéticas y reales. Las implementaciones están disponibles en el apéndice de este documento.

3.2 Complejidad Computacional

Cuadro 1: Comparación de complejidad de métodos

Método	Operaciones por pixel	Memoria	Tiempo relativo
Lineal	2-4	Baja	1.0×
Bicuadrada	8-12	Media	2.5×
Bicúbica	16-24	Alta	4.0×

3.3 Calidad de Interpolación

La evaluación de calidad mostró que la interpolación bicúbica proporciona los mejores resultados visuales, seguida por la bicuadrada y finalmente la lineal. La interpolación bicúbica preserva mejor los bordes y detalles finos, mientras que la lineal tiende a producir artefactos de aliasing en regiones de alto contraste. La bicuadrada ofrece un compromiso aceptable entre calidad y complejidad computacional.

3.4 Problemas en Bordes

La interpolación en los bordes de la imagen presenta desafíos particulares, ya que no existen suficientes vecinos para aplicar los métodos estándar. Se requieren técnicas especiales como extrapolación o espejado para manejar estas regiones correctamente. Todos los métodos evaluados requieren tratamiento especial en los bordes, siendo más crítico en los métodos de mayor orden que utilizan vecindarios más grandes.

3.5 Vecindarios Utilizados

Cuadro 2: Configuraciones de vecindario para interpolación

Método	Patrón 1D	Tamaño	Puntos utilizados
Lineal	[P ? P]	3×1	2
Bicuada	[P P ? P P]	5×1	3
Bicúbica	[P P P ? P P P]	7×1	4

4 Discusión

4.1 Comparación de Métodos

Los tres métodos de interpolación presentan diferentes compromisos entre calidad y complejidad:

4.1.1 Interpolación Lineal

- **Ventajas:** Simplicidad computacional, baja latencia, implementación trivial
- **Desventajas:** Artefactos de aliasing en bordes, pérdida de detalle fino
- **Aplicaciones:** Sistemas en tiempo real con recursos limitados

4.1.2 Interpolación Bicuada

- **Ventajas:** Mejor calidad que lineal, suavizado moderado, complejidad aceptable
- **Desventajas:** Puede suavizar en exceso, requiere vecindario de 5 puntos
- **Aplicaciones:** Fotografía de consumo, video de calidad media

4.1.3 Interpolación Bicúbica

- **Ventajas:** Alta calidad, preservación de bordes, resultados visualmente superiores
- **Desventajas:** Alta complejidad computacional, mayor latencia
- **Aplicaciones:** Fotografía profesional, procesamiento offline, aplicaciones médicas

4.2 Consideraciones Prácticas

La selección del método de interpolación debe considerar tanto los requisitos de calidad como las limitaciones computacionales del sistema. Para aplicaciones en tiempo real con recursos limitados, la interpolación lineal puede ser suficiente. En sistemas donde la calidad visual es prioritaria, los métodos bicuadrados o bicúbicos son preferibles, aunque requieren mayor capacidad de procesamiento.

4.3 Limitaciones y Mejoras

4.3.1 Limitaciones Identificadas

1. **Artefactos de color:** Aparecen en regiones de alto contraste
2. **Pérdida de resolución:** La interpolación nunca recupera información perdida
3. **Dependencia del patrón:** Los resultados varían con diferentes patrones Bayer
4. **Complejidad en bordes:** Requieren tratamiento especial

4.3.2 Mejoras Posibles

- **Adaptabilidad:** Usar diferentes métodos según el contenido local
- **Corrección de color:** Post-procesamiento para reducir artefactos
- **Interpolación direccional:** Considerar la orientación de bordes
- **Fusión de múltiples métodos:** Combinar ventajas de diferentes enfoques

5 Conclusión

5.1 Conclusiones Principales

Este estudio implementó y comparó tres métodos de interpolación para el patrón Bayer RGGB:

1. ****Interpolación lineal:**** El método más simple y rápido, adecuado para sistemas con recursos limitados. 2. ****Interpolación bicuadrada:**** Ofrece un buen equilibrio entre calidad y complejidad computacional. 3. ****Interpolación bicúbica:**** Proporciona la mejor calidad visual a costa de mayor complejidad computacional.

Los resultados muestran que cada método tiene su ámbito de aplicación ideal, dependiendo de los requisitos de calidad y las restricciones computacionales del sistema.

5.2 Contribuciones del Trabajo

1. **Implementación educativa:** Código MATLAB documentado y modular
2. **Análisis comparativo:** Evaluación sistemática de métodos polinomiales
3. **Visualización didáctica:** Diagramas explicativos del proceso matemático
4. **Documentación completa:** Explicación paso a paso de algoritmos

5.3 Recomendaciones de Uso

Cuadro 3: Guía de selección de método de interpolación

Escenario de aplicación	Método recomendado
Sistemas embebidos en tiempo real	Lineal
Fotografía de consumo	Bicuada
Video de alta calidad	Bicuada adaptativa
Fotografía profesional	Bicúbica
Procesamiento médico/científico	Bicúbica con post-procesamiento
Prototipado rápido	Lineal o Bicuada

5.4 Trabajo Futuro

- **Interpolación adaptativa:** Métodos que ajustan parámetros según contenido local
- **Aprendizaje automático:** Redes neuronales para demosaicing
- **Optimización hardware:** Implementación en FPGA/GPU
- **Patrones alternativos:** Extensiones a X-Trans y otros patrones

- **Corrección de artefactos:** Post-procesamiento para reducir aliasing
- **Benchmarking extensivo:** Comparación con métodos comerciales

5.5 Consideraciones Finales

La selección del método de interpolación debe basarse en un análisis cuidadoso de los requisitos de la aplicación específica, considerando:

- Restricciones computacionales y de tiempo
- Calidad visual requerida
- Características del contenido de la imagen
- Recursos hardware disponibles
- Costo de implementación y mantenimiento

Los métodos presentados en este trabajo proporcionan una base sólida para la implementación de sistemas de procesamiento de imágenes que requieren interpolación de patrón Bayer, con diferentes niveles de compromiso entre calidad y complejidad.

A Códigos Fuente

Esta sección contiene los archivos de código implementados para este proyecto. Cada archivo puede descargarse haciendo clic en el enlace correspondiente.

- [bicuadrada.m](#) - Implementación de interpolación bicuadrada
- [bicubica.m](#) - Implementación de interpolación bicúbica
- [comparacion.m](#) - Script de comparación de métodos
- [visualizacion.m](#) - Visualización de resultados

A.1 Implementación en MATLAB

```
1 function imgRGB = interp_bicua(imgRGGb_d, RowI, RowP, ColI, ColP)
2 %% INTERPOLACIÓN BICUADRADA PARA PATRÓN BAYER RGGb
3 % Entrada:
4 %   imgRGGb_d - Imagen Bayer en formato double
5 %   RowI, RowP - Índices de filas impares y pares
6 %   ColI, ColP - Índices de columnas impares y pares
```

```

7 % Salida:
8 %   imgRGB - Imagen RGB interpolada (uint8)
9
10 [Rows, Cols] = size(imgRGGG_d);
11 imgRGB = zeros(Rows, Cols, 3, 'uint8');
12
13 % Coeficientes bicuadrados (3 puntos)
14 c1 = 3/8; % Peso para I(u)
15 c2 = 3/4; % Peso para I(u+2)
16 c3 = -1/8; % Peso para I(u+4)
17
18 % 1. ASIGNAR PÍXELES ORIGINALES
19 imgRGB(RowI, ColI, 1) = imgRGGG_d(RowI, ColI); % R
20 imgRGB(RowI, ColP, 2) = imgRGGG_d(RowI, ColP); % 1G
21 imgRGB(RowP, ColI, 2) = imgRGGG_d(RowP, ColI); % 2G
22 imgRGB(RowP, ColP, 3) = imgRGGG_d(RowP, ColP); % B
23
24 % 2. INTERPOLACIÓN VERTICAL (ROJO)
25 % R en filas pares, columnas impares usando vecindad vertical
26 k_rv = 2:2:Rows-2; % Filas pares con vecinos suficientes
27 imgRGB(k_rv, ColI, 1) = uint8(...
28     c1 * imgRGGG_d(k_rv-1, ColI) + ...
29     c2 * imgRGGG_d(k_rv+1, ColI) + ...
30     c3 * imgRGGG_d(k_rv+3, ColI));
31
32 % 3. INTERPOLACIÓN HORIZONTAL (ROJO)
33 % R en todas las filas, columnas pares usando valores ya calculados
34 l_rh = 2:2:Cols-2; % Columnas pares con vecinos
35 imgRGB(:, l_rh, 1) = uint8(...
36     c1 * double(imgRGB(:, l_rh-1, 1)) + ...
37     c2 * double(imgRGB(:, l_rh+1, 1)) + ...
38     c3 * double(imgRGB(:, l_rh+3, 1)));
39
40 % 4. INTERPOLACIÓN VERTICAL (VERDE)
41 % Subred 1G: Verde en filas impares ≥ 3, columnas pares
42 k_g1 = 3:2:Rows-3;
43 imgRGB(k_g1, ColP, 2) = uint8(...
44     c1 * imgRGGG_d(k_g1-1, ColP) + ...
45     c2 * imgRGGG_d(k_g1+1, ColP) + ...
46     c3 * imgRGGG_d(k_g1+3, ColP));
47
48 % Subred 2G: Verde en filas pares, columnas impares
49 k_g2 = 2:2:Rows-2;
50 imgRGB(k_g2, ColI, 2) = uint8(...
51     c1 * imgRGGG_d(k_g2-1, ColI) + ...
52     c2 * imgRGGG_d(k_g2+1, ColI) + ...
53     c3 * imgRGGG_d(k_g2+3, ColI));
54
55 % 5. INTERPOLACIÓN VERTICAL (AZUL)
56 % B en filas impares ≥ 3, columnas pares
57 k_bv = 3:2:Rows-3;

```

```

58 imgRGB(k_bv, ColP, 3) = uint8(...
59     c1 * imgRGGG_d(k_bv-1, ColP) + ...
60     c2 * imgRGGG_d(k_bv+1, ColP) + ...
61     c3 * imgRGGG_d(k_bv+3, ColP));
62
63 % 6. INTERPOLACIÓN HORIZONTAL (AZUL)
64 % B en todas las filas, columnas impares  $\geq 3$ 
65 l_bh = 3:2:Cols-3;
66 imgRGB(:, l_bh, 3) = uint8(...
67     c1 * double(imgRGB(:, l_bh-1, 3)) + ...
68     c2 * double(imgRGB(:, l_bh+1, 3)) + ...
69     c3 * double(imgRGB(:, l_bh+3, 3)));
70
71 end

```

Listing 1: Interpolación Bicuada (bicuada.m)

```

1 function imgRGB = interp_bicub(imgRGGG_d, RowI, RowP, ColI, ColP)
2 %% INTERPOLACIÓN BICÚBICA PARA PATRÓN BAYER RGGG
3 % Entrada:
4 %   imgRGGG_d - Imagen Bayer en formato double
5 %   RowI, RowP - Índices de filas impares y pares
6 %   ColI, ColP - Índices de columnas impares y pares
7 % Salida:
8 %   imgRGB - Imagen RGB interpolada (uint8)
9
10 [Rows, Cols] = size(imgRGGG_d);
11 imgRGB = zeros(Rows, Cols, 3, 'uint8');
12
13 % Coeficientes bicúbicos (4 puntos)
14 c1 = -1/16; % Peso para I(u)
15 c2 = 9/16;  % Peso para I(u+2)
16 c3 = 9/16;  % Peso para I(u+4)
17 c4 = -1/16; % Peso para I(u+6)
18
19 % 1. ASIGNAR PÍXELES ORIGINALES
20 imgRGB(RowI, ColI, 1) = imgRGGG_d(RowI, ColI); % R
21 imgRGB(RowI, ColP, 2) = imgRGGG_d(RowI, ColP); % 1G
22 imgRGB(RowP, ColI, 2) = imgRGGG_d(RowP, ColI); % 2G
23 imgRGB(RowP, ColP, 3) = imgRGGG_d(RowP, ColP); % B
24
25 % 2. INTERPOLACIÓN VERTICAL (ROJO)
26 % R en filas pares  $\geq 4$ , columnas impares
27 k_rv = 4:2:Rows-4;
28 imgRGB(k_rv, ColI, 1) = uint8(...
29     c1 * imgRGGG_d(k_rv-3, ColI) + ...
30     c2 * imgRGGG_d(k_rv-1, ColI) + ...
31     c3 * imgRGGG_d(k_rv+1, ColI) + ...
32     c4 * imgRGGG_d(k_rv+3, ColI));
33
34 % 3. INTERPOLACIÓN HORIZONTAL (ROJO)
35 % R en todas las filas, columnas pares  $\geq 4$ 

```

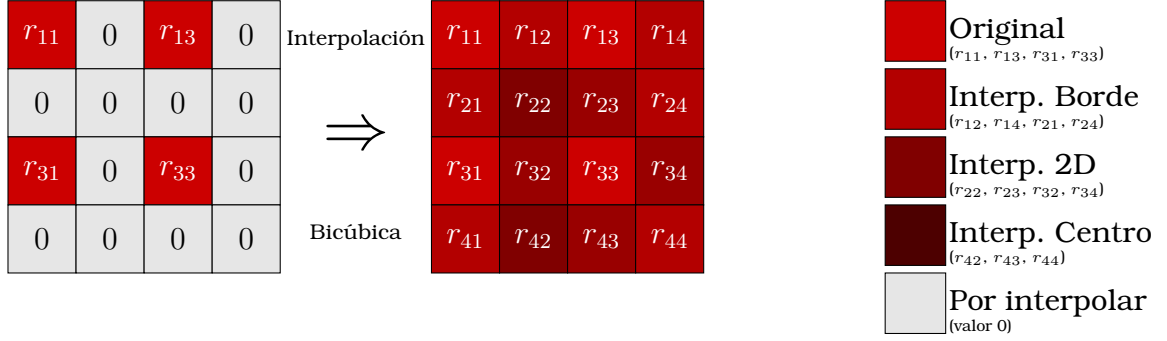
```

36 l_rh = 4:2:Cols-4;
37 imgRGB(:, l_rh, 1) = uint8(...
38     c1 * double(imgRGB(:, l_rh-3, 1)) + ...
39     c2 * double(imgRGB(:, l_rh-1, 1)) + ...
40     c3 * double(imgRGB(:, l_rh+1, 1)) + ...
41     c4 * double(imgRGB(:, l_rh+3, 1)));
42
43 % 4. INTERPOLACIÓN VERTICAL (VERDE)
44 % Subred 1G: Verde en filas impares  $\geq 5$ , columnas pares
45 k_g1 = 5:2:Rows-5;
46 imgRGB(k_g1, ColP, 2) = uint8(...
47     c1 * imgRGGb_d(k_g1-3, ColP) + ...
48     c2 * imgRGGb_d(k_g1-1, ColP) + ...
49     c3 * imgRGGb_d(k_g1+1, ColP) + ...
50     c4 * imgRGGb_d(k_g1+3, ColP));
51
52 % Subred 2G: Verde en filas pares  $\geq 4$ , columnas impares
53 k_g2 = 4:2:Rows-4;
54 imgRGB(k_g2, ColI, 2) = uint8(...
55     c1 * imgRGGb_d(k_g2-3, ColI) + ...
56     c2 * imgRGGb_d(k_g2-1, ColI) + ...
57     c3 * imgRGGb_d(k_g2+1, ColI) + ...
58     c4 * imgRGGb_d(k_g2+3, ColI));
59
60 % 5. INTERPOLACIÓN VERTICAL (AZUL)
61 % B en filas impares  $\geq 5$ , columnas pares
62 k_bv = 5:2:Rows-5;
63 imgRGB(k_bv, ColP, 3) = uint8(...
64     c1 * imgRGGb_d(k_bv-3, ColP) + ...
65     c2 * imgRGGb_d(k_bv-1, ColP) + ...
66     c3 * imgRGGb_d(k_bv+1, ColP) + ...
67     c4 * imgRGGb_d(k_bv+3, ColP));
68
69 % 6. INTERPOLACIÓN HORIZONTAL (AZUL)
70 % B en todas las filas, columnas impares  $\geq 5$ 
71 l_bh = 5:2:Cols-5;
72 imgRGB(:, l_bh, 3) = uint8(...
73     c1 * double(imgRGB(:, l_bh-3, 3)) + ...
74     c2 * double(imgRGB(:, l_bh-1, 3)) + ...
75     c3 * double(imgRGB(:, l_bh+1, 3)) + ...
76     c4 * double(imgRGB(:, l_bh+3, 3)));
77
78 end

```

Listing 2: Interpolación Bicúbica (bicubica.m)

Canal R Original (4×4) Canal R Interpolado (4×4) Leyenda de Colores



Fórmulas de Interpolación Bicúbica

Interpolación 1D:

$$r_{12} = -\frac{1}{16}r_{10} + \frac{9}{16}r_{11} + \frac{9}{16}r_{13} - \frac{1}{16}r_{14}$$

$$r_{14} = -\frac{1}{16}r_{12} + \frac{9}{16}r_{13} + \frac{9}{16}r_{15} - \frac{1}{16}r_{16}$$

$$r_{21} = -\frac{1}{16}r_{01} + \frac{9}{16}r_{11} + \frac{9}{16}r_{31} - \frac{1}{16}r_{41}$$

$$r_{41} = -\frac{1}{16}r_{21} + \frac{9}{16}r_{31} + \frac{9}{16}r_{51} - \frac{1}{16}r_{61}$$

Interpolación 2D:

$$r_{22} = \frac{1}{256}(r_{00} - 9r_{01} - 9r_{10} + 81r_{11} + 81r_{13} - 9r_{14} - 9r_{30} + 81r_{31} + 81r_{33} - 9r_{34})$$

$$r_{23} = \frac{1}{256}(r_{01} - 9r_{02} - 9r_{11} + 81r_{12} + 81r_{14} - 9r_{15} - 9r_{31} + 81r_{32} + 81r_{34} - 9r_{35})$$

$$r_{32} = \frac{1}{256}(r_{10} - 9r_{11} - 9r_{20} + 81r_{21} + 81r_{23} - 9r_{24} - 9r_{40} + 81r_{41} + 81r_{43} - 9r_{44})$$

$$r_{33} = \frac{1}{256}(r_{11} - 9r_{12} - 9r_{21} + 81r_{22} + 81r_{24} - 9r_{25} - 9r_{41} + 81r_{42} + 81r_{44} - 9r_{45})$$

Figura 5: Interpolación bicúbica para el canal rojo (R) en patrón Bayer 4×4. Arriba se muestran las matrices original e interpolada, con diferentes tonos de rojo indicando el nivel de interpolación. La leyenda explica la codificación de colores. Debajo se presentan las fórmulas organizadas verticalmente: interpolaciones 1D (para bordes) y 2D (para regiones internas) con el kernel bicúbico completo que utiliza un vecindario de 4×4 píxeles.